e Classification

README • Code walkthrough • Setup requirements

A beginner-friendly guide to the ResNet50 + FastAPI

Project at a glance

GOAL	INPUT	OUTPUT	TECHNOLOGY
Classify an image into one of five flower classes.	An uploaded image file.	Predicted class, confidence, and top-three results.	Python, TensorFlow/Keras, ResNet50, FastAPI, Pillow, NumPy.

Important: The notebook contains code, but the trained model is a separate file named `flower_resnet50.keras`. To run the API, keep that model beside `app.py` and use the model path described below.

1. Project overview

This project classifies flower images into five categories. A ResNet50 convolutional neural network pre-trained on ImageNet is used as a feature extractor. A custom classification head is added for the flower classes. The trained model is then loaded by a FastAPI web service so users can upload an image and receive a prediction.

The five classes are:

- dandelion
- daisy
- tulips
- sunflowers
- roses

Expected project files

File	Purpose
training_notebook.ipynb	Colab notebook: dataset preparation, model building, training, evaluation, and saving.
app.py	FastAPI application that loads the model and exposes prediction endpoints.
flower_resnet50.keras	Saved trained Keras model required at runtime.
requirements.txt	Python packages needed to run the API.
README.md / PDF guide	Project description, setup instructions, and code explanation.

2. What the training code does

Step 1 — Load the dataset

The notebook loads a flower image dataset and organizes examples into training, validation, and test sets. Recorded split sizes were 2,569 training images, 551 validation images, and 550 test images. Training data teaches the model; validation helps monitor training; test data checks performance on unseen examples.

Step 2 — Prepare images

Images are resized to 224 × 224 pixels to match ResNet50 input size. ResNet50 preprocessing adjusts RGB pixel values to the format expected by its ImageNet weights.

Step 3 — Use transfer learning

ResNet50 is loaded with ImageNet pre-trained weights and its original classifier is excluded. The base network is frozen during initial training so the project can reuse features learned from a large image collection.

Step 4 — Add a flower classification head

The model adds GlobalAveragePooling2D, Dropout(0.3), Dense(128, activation="relu"), Dropout(0.2), and a final Dense layer with five softmax outputs. The output scores correspond to the five flower classes.

Step 5 — Train and validate

The model learns from the training split and is monitored with validation data. Confirm epochs, batch size, optimizer, and loss from the final notebook because these settings can vary.

Step 6 — Evaluate and save

The test split checks performance on data not used for fitting. Save the model as flower_resnet50.keras. The model is a separate file and is not automatically bundled into a downloaded notebook.

3. What app.py does

Code part	What it does
Imports	Loads FastAPI, TensorFlow, NumPy, Pillow image utilities, and ResNet50 preprocessing support.
FastAPI app	Creates the web API and its title/description.
Load model	Loads flower_resnet50.keras when the API starts. The preprocessing function is registered for model deserialization.
GET /	Returns a message confirming the API is running.
POST /predict	Accepts an image upload, checks it can be opened, converts it to RGB, resizes to 224 × 224, and runs prediction.
Prediction ranking	Sorts model scores and returns the highest-scoring flower, confidence percentage, and top three classes.
Error handling	Returns HTTP 400 when the uploaded file is not a valid image.

Preprocessing note: The described model already includes ResNet50 preprocessing as a layer. Do not apply `preprocess_input` a second time in `/predict`; pass the resized RGB image array to the model.

Example response from POST /predict

```
{<br/> "prediction": "roses",<br/> "confidence": 98.8,<br/> "top_3": [<br/> {"flower": "roses",
"confidence": 98.8},<br/> {"flower": "tulips", "confidence": 1.2},<br/> {"flower": "dandelion",
"confidence": 0.0}<br/> ]<br/>}
```

Example only. Actual predictions depend on the uploaded image and model.

4. Requirements

Requirement	Purpose / notes
Python 3.10 or 3.11 (recommended starting point)	Choose a Python version compatible with the TensorFlow release installed.
TensorFlow	Loads the model and runs predictions.
FastAPI	Defines the HTTP API routes.
Uvicorn	Runs the FastAPI server.
python-multipart	Enables file uploads.
NumPy	Handles image arrays and ranks predictions.
Pillow	Opens, converts, and resizes images.
flower_resnet50.keras	Saved model must exist at the path configured in app.py.

Example requirements.txt

```
tensorflow<br/>fastapi<br/>uvicorn[standard]<br/>python-multipart<br/>numpy<br/>Pillow
```

These package names are not version-pinned. If installation fails, choose a Python/TensorFlow combination supported by the TensorFlow installation documentation.

5. Step-by-step: run the API on Windows

1. Create/open the project folder

D:\AI Training Projects\Task 1

2. Put the files together

Copy app.py, flower_resnet50.keras, and requirements.txt into the folder. Keep the downloaded .ipynb notebook there too.

3. Open Command Prompt in the folder

In File Explorer, open the folder, click the address bar, type cmd, and press Enter.

4. Create a virtual environment

Run: python -m venv .venv

5. Activate it

Run: .venv\Scripts\activate

6. Install packages

Run: python -m pip install --upgrade pip, then pip install -r requirements.txt

7. Check the model path

Use a path relative to app.py, not the Colab-only /content/ path. Keep the model in the same folder as app.py.

8. Start the server

Run: uvicorn app:app --reload

9. Open API documentation

Visit http://127.0.0.1:8000/docs in your browser.

10. Test prediction

Expand POST /predict, click Try it out, choose an image file, then click Execute.

6. Important model-path adjustment

The original Colab path /content/flower_resnet50.keras only exists in Colab. For Windows, use a path relative to app.py:

```
from pathlib import Path<br/>MODEL_PATH = Path(__file__).resolve().parent /
"flower_resnet50.keras"<br/><br/>model = tf.keras.models.load_model(<br/> MODEL_PATH,<br/>
custom_objects={"preprocess_input": preprocess_input},<br/> compile=False<br/>)
```

Keep the rest of app.py consistent with the final notebook. If the model file is missing, the API cannot start; re-run the training/saving steps or restore the model file first.

7. Common issues and fixes

Issue	What to check
Model file not found	Confirm flower_resnet50.keras is beside app.py and the path is correct.
Missing Python package	Activate the intended virtual environment and install requirements.txt.
Upload fails	Confirm python-multipart is installed and test in /docs.
Invalid image message	Try a valid JPG or PNG image.
Prediction seems wrong	Check the model file, class order, image size, and duplicate preprocessing.
Notebook has no model	Notebook download and model download are separate tasks. Save the .keras file separately.

8. Submission checklist

- ■ Downloaded training notebook (.ipynb).
- ■ app.py included.
- ■ flower_resnet50.keras included (required to run API).
- ■ requirements.txt included.
- ■ README documentation included.
- ■ Tested GET / and POST /predict in FastAPI docs.
- ■ Project folder zipped if the LMS requires one upload.

Suggested final folder

```
Task 1/<br/>■■■ training_notebook.ipynb<br/>■■■ app.py<br/>■■■ flower_resnet50.keras<br/>■■■ requirements.txt<br/>■■■ README.md
```

End of guide